EXPERIMENT – 3: INFORMATION THEORY USING PLAY TENNIS DATASET

AIM

To calculate the **Entropy** and **Information Gain** using the **Play Tennis Dataset** and identify the best attribute (root node) for constructing a Decision Tree.

THEORY

Information Theory is used in Machine Learning to measure the uncertainty of a dataset. It is widely used in Decision Tree algorithms such as **ID3**.

- **Entropy** measures the impurity or randomness in the dataset.
- **Information Gain** measures the reduction in entropy after splitting the dataset based on an attribute.
- The attribute with the **highest Information Gain** is selected as the **Root Node** of the Decision Tree.

ALGORITHM

1. Import the required libraries.
2. Create the Play Tennis dataset.
3. Calculate the entropy of the target attribute (**PlayTennis**).
4. Calculate the Information Gain for each input attribute.
5. Compare the Information Gain values.
6. Select the attribute with the highest Information Gain.
7. Display the entropy, information gain values, and best attribute.

PROGRAM

```python
import pandas as pd

from math import log2


# Play Tennis Dataset

data = {

    "Outlook": ["Sunny","Sunny","Overcast","Rain","Rain","Rain","Overcast",

                "Sunny","Sunny","Rain","Sunny","Overcast","Overcast","Rain"],

    "Temperature": ["Hot","Hot","Hot","Mild","Cool","Cool","Cool",

                    "Mild","Cool","Mild","Mild","Mild","Hot","Mild"],

    "Humidity": ["High","High","High","High","Normal","Normal","Normal",

                 "High","Normal","Normal","Normal","High","Normal","High"],

    "Wind": ["Weak","Strong","Weak","Weak","Weak","Strong","Strong",

             "Weak","Weak","Weak","Strong","Strong","Weak","Strong"],

    "PlayTennis": ["No","No","Yes","Yes","Yes","No","Yes",

                   "No","Yes","Yes","Yes","Yes","Yes","No"]

}


df = pd.DataFrame(data)


# Function to calculate Entropy

def entropy(target):

    values = target.value_counts(normalize=True)

    return -sum(p * log2(p) for p in values)
```

```
# Function to calculate Information Gain
def information_gain(df, attribute, target):
    total_entropy = entropy(df[target])

    weighted_entropy = 0

    for value in df[attribute].unique():
        subset = df[df[attribute] == value]
        weighted_entropy += (len(subset) / len(df)) * entropy(subset[target])

    return total_entropy - weighted_entropy

# Entropy
target_entropy = entropy(df["PlayTennis"])

print("Entropy of PlayTennis:", round(target_entropy, 4))

print("\nInformation Gain:")

gains = {}

for column in df.columns[:-1]:
    gain = information_gain(df, column, "PlayTennis")
    gains[column] = gain
```

```
    print(f"{column}: {gain:.4f}")

best_attribute = max(gains, key=gains.get)

print("\nBest Attribute (Root Node):", best_attribute)
```

DATASET

Dataset Name: Play Tennis Dataset

- Total Records : **14**
- Input Attributes : **4**
  - Outlook
  - Temperature
  - Humidity
  - Wind
- Target Attribute : **PlayTennis**
- Target Classes :
  - Yes
  - No

OUTPUT

Entropy of PlayTennis: 0.9403

Information Gain:

Outlook: 0.2467

Temperature: 0.0292

Humidity: 0.1518

Wind: 0.0481

Best Attribute (Root Node): Outlook

CALCULATION

Entropy of Target Attribute

Target	Entropy
PlayTennis	**0.9403**

Information Gain

Attribute	Information Gain
Outlook	**0.2467**
Temperatue	**0.0292**
Humidity	**0.1518**
Wind	**0.0481**

Highest Information Gain = **0.2467**

Best Attribute = Outlook

RESULT

Thus, the **Entropy** and **Information Gain** of the **Play Tennis Dataset** were successfully calculated. The entropy of the target attribute (**PlayTennis**) is **0.9403**. Among all the input attributes, **Outlook** has the highest Information Gain (**0.2467**); therefore, **Outlook** is selected as the **Root Node** for constructing the Decision Tree.